

TP5 : neural network - Part 2 -

In the last tutorial, we have modeled a three layers neural network as follows:

- $a^{(1)} = X$
- $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$
- $a^{(2)} = \sigma(z^{(2)})$
- $z^{(3)} = W^{(3)}a^{(2)} + b^{(3)}$
- $a^{(3)} = \sigma(z^{(3)})$

This model was used to solve a binary classification problem. Two different formulations were tested: at first, the output $a^{(3)}$ was set as a real ($a^{(3)} \in [0, 1]$ due to the sigmoid function) and then, in a second experiment $a^{(3)} = t$ ($(a^{(3)})_1, (a^{(3)})_2 \in [0, 1] \times [0, 1]$). This last formulation can easily be generalized to do multiclass classification instead of binary classification. However, applying a sigmoid function at the output vector of the model is not necessarily the best option. Ideally, one would like to have an output probability vector, i.e. a vector that satisfies $(a^{(3)})_1 + (a^{(3)})_2 = 1$. We can make sure that $a^{(3)}$ is a probability vector by replacing the sigmoid function σ of the last layer by the softmax function $s : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ defined by

$$s(z) = \begin{pmatrix} \frac{e^{z_1}}{e^{z_1} + e^{z_2}} \\ \frac{e^{z_2}}{e^{z_1} + e^{z_2}} \end{pmatrix}$$

It is easy to see that $\forall z = (z_1, z_2) \in \mathbb{R}^2$, $\frac{e^{z_1}}{e^{z_1} + e^{z_2}} + \frac{e^{z_2}}{e^{z_1} + e^{z_2}} = 1$ and therefore $s(z)$ is a probability vector. In the first part of this tutorial, you will have to modify the binary classification neural network model by replacing the sigmoid function of the last layer by the softmax function. We will stick to the binary cross entropy cost function to find the parameters of the model

$$C = \frac{-1}{m} \sum_{k=1}^2 \sum_x y_k(x) \ln((a_x^{(3)})_k) + (1 - y_k(x)) \ln(1 - (a_x^{(3)})_k),$$

however, when the last layer is a probability vector, i.e. $(a^{(3)})_1 + (a^{(3)})_2 = 1$, we can simplify this expression.

Question 1:

Show that the above expression simplifies to

$$C = \frac{-2}{m} \sum_{k=1}^2 \sum_x y_k(x) \ln((a_x^{(3)})_k).$$

Hint : remember that we also have $y_1(x) + y_2(x) = 1$.

In practice, the coefficient 2 appearing in this cost function does not change the location of the minimum and it is usually removed so that the cost function becomes

$$C = \frac{1}{m} \sum_{k=1}^2 \sum_x y_k(x) \ln((a_x^{(3)})_k).$$

Here again, the part of the backpropagation algorithm that needs to be modified is

$$\Delta^{(3)}_{def} = \frac{\partial C}{\partial z^{(3)}} = \begin{pmatrix} \frac{\partial C}{\partial (z^{(3)})_1} \\ \frac{\partial C}{\partial (z^{(3)})_2} \end{pmatrix}$$

Question 2:

First, show that

$$C = \frac{1}{m} \sum_x \ln(e^{(z_x^{(3)})_1} + e^{(z_x^{(3)})_2}) - y_1(x)(z_x^{(3)})_1 - y_2(x)(z_x^{(3)})_2,$$

and then

$$\Delta^{(3)} = \begin{pmatrix} (a_x^{(3)})_1 - y_1(x) \\ (a_x^{(3)})_2 - y_2(x) \end{pmatrix}$$

When using a binary cross entropy cost function, is there any difference in the computation of $\Delta^{(3)}$ with a sigmoid function or a softmax activation function in the last layer?

Question 3:

Starting from your python code that implement the sinus/exponentiel classification with a two dimensional output vector and a sigmoid activation function, make the necessary modifications so that it now use a softmax activation function in the last layer. The parts of the code that need to be modified are listed below:

- create a softmax function
- modify the backpropagation algorithm
- modify the *model_predict* function
- modify the cost function so that it uses the simplified formula

We are now ready to build a model to solve an important problem : image classification. For this exercise, you will use a neural network to recognize handwritten digits (from 0 to 9). Automated handwritten digit recognition is widely used today - from recognizing zip codes (postal codes) on mail envelopes

to recognizing amounts written on bank checks. This exercise will show you how the methods you have learned can be used for this classification task.

You are given a data set in the file *ex3data1.mat* that contains 5000 training examples of handwritten digits. The ‘.mat’ format means that the data has been saved in a native Octave/MATLAB matrix format, instead of a text (ASCII) format like a csv-file. Fortunately, python provides mechanisms to load MATLAB native format using the *loadmat* function within the *scipy.io* module. This function returns a python dictionary with keys containing the variable names within the .mat file.

There are $m = 5000$ training examples in *ex3data1.mat*, where each training example is a 20 pixel by 20 pixel grayscale image of the digit. Each pixel is represented by a floating point number indicating the grayscale intensity at that location. The 20 by 20 grid of pixels is “unrolled” into a 400-dimensional vector. Each of these training examples becomes a single column in our data matrix X . This gives us a 400 by 5000 matrix X where every column is a training example for a handwritten digit image:

$$X = \begin{pmatrix} | & & | \\ x_1 & \cdots & x_m \\ | & & | \end{pmatrix}$$

The second part of the training set is a 5000-dimensional vector Y that contains labels for the training set:

$$Y = (y_1 \quad \cdots \quad y_m),$$

where $y_i \in [0, 9]$ indicates the digit on the image x_i .

The jupyter notebook provided starts by loading the dataset. The next couple of cells allows you to visualize a subset of the training set. The code randomly selects 100 images from X and passes those rows to the *displayData* function. This function maps each row to a 20 pixel by 20 pixel grayscale image and displays the images together. You can execute the cells to see what typical images look like. Instead of having two classes as with the sinus/exponential classification problem, we now have ten classes. The cost function we’ll use is a generalization of the binary cross entropy we have introduced at the beginning of this tutorial. Here it will take the form

$$C = \frac{-1}{m} \sum_{k=1}^{10} \sum_x (y(x))_k \ln((a_x^{(3)})_k),$$

which requires a modification of the coding of the output Y . Indeed, in the above formula, the activation function is assumed to be the softmax function that will return a probability vector of size 10. Therefore, the scalar $y_i = y(x_i) = d \in [0, 9]$ will have to be transformed into a vector of size 10 having zeros everywhere except at position p where there will be a one. Such encoding is

very common and it is called "one hot encoding". As a consequence, Y will be a matrix of size 10 by 5000

$$Y = \begin{pmatrix} | & & | \\ y_1 & \cdots & y_m \\ | & & | \end{pmatrix}$$

Question 4:

Specify the dimensions of the matrices/vectors $W^{(2)}$, $W^{(3)}$, $b^{(2)}$ and $b^{(3)}$. Using the binary classification code of this tutorial and the provided code that read the data, make the necessary modifications so that the neural network can classify digits on the images. The parts of the code that need to be modified are listed below:

- read the data + one hot encode the y_i outputs with the provided code
- set the new shapes of the matrices/vectors $W^{(2)}$, $W^{(3)}$, $b^{(2)}$ and $b^{(3)}$.
- modify the backpropagation algorithm
- modify the computation of the accuracy, if necessary

What you should observe is that your model is very good at classifying digits that were used to build the model. However, what's really important is that your model be able to accurately classify images it has never seen before. To test that, we will slightly modify the setting of the problem: we will build the model with only 4000 images (instead of 5000) and the remaining 1000 images will serve to test the model in real conditions.

Question 5:

Before building two separate dataset (one for training and one for testing), you must start by shuffling the data. Why do you need to do that? To shuffle the data, you can use the *shuffle* function from the library *sklearn.utils*. After separating the training data (into X and Y matrices) and testing data (into X_{test} and Y_{test} matrices), what will be the size of these matrices? Modify the code so that it trains on the 4000 images dataset. Compare the accuracy on the training dataset and on the testing dataset.